

What is a Hamiltonian Cycle?

In graph theory, a **Hamiltonian Cycle** (also called a Hamiltonian circuit) is a closed loop in a graph that visits every vertex exactly once and returns to the starting point. A graph that contains at least one such cycle is known as a **Hamiltonian graph**.

Key Characteristics

- **Vertex Coverage:** Every single vertex in the graph must be visited.
- **Unique Visits:** Each vertex is visited exactly once, except for the starting vertex which is also the ending vertex.
- **Edges:** Unlike an Eulerian circuit (which must use every *edge*), a Hamiltonian cycle does not need to use every edge in the graph.
- **Difference from Hamiltonian Path:** A Hamiltonian *path* also visits every vertex once, but it is an "open" path—it does not need to return to the starting vertex.

Practical Applications

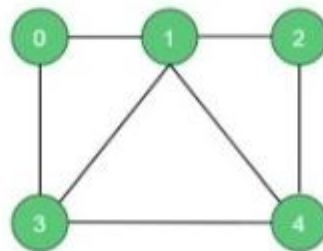
Hamiltonian cycles are not just theoretical concepts; they are used to solve complex real-world optimization problems:

- **Traveling Salesman Problem (TSP):** Finding the Hamiltonian cycle with the least total weight (cost) in a weighted graph.
- **Logistics and Routing:** Planning delivery routes where a driver must visit a set of locations exactly once and return to the depot.

Input Output Scenario

Suppose the given undirected graph $G(V, E)$ and its adjacency matrix are as follows

0	1	0	1	0
1	0	1	1	1
0	1	0	0	1
1	1	0	0	1
0	1	1	1	0



Backtracking is a systematic method for solving problems by exploring all potential possibilities. It is essentially a refined version of **Brute Force** that builds a solution incrementally and abandons paths that fail to meet the problem's constraints.

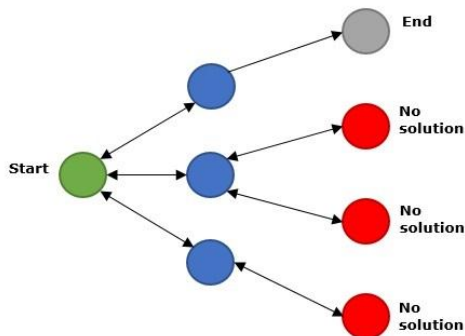
The Core Logic

1. **Selection:** You make a choice from a set of available options at the current step.
2. **Validation:** You check if this choice violates any rules (constraints).
3. **Recursion:** If valid, you move forward to the next step.

4. **Backtrack:** If the choice leads to a "dead end" where no further valid moves are possible, you **go back** to the previous step, undo the choice, and try the next available option.

Key Characteristics

- **State Space Tree:** The algorithm explores the problem as a tree where each node is a state and branches are choices.
- **Pruning:** It saves time by "pruning" branches that are guaranteed not to lead to a solution, rather than checking every single possibility.
- **Recursive:** It is almost always implemented using recursion to keep track of previous states.



The **N-Queens problem** is a classic combinatorial puzzle in computer science and mathematics. It serves as a benchmark for evaluating search algorithms and backtracking techniques.

Definition

The goal is to place N chess queens on an N X N chessboard so that no two queens threaten each other. In chess, a queen can attack horizontally, vertically, or diagonally. Therefore, a valid solution requires:

- **No two queens in the same row.**
- **No two queens in the same column.**
- **No two queens on the same diagonal** (both main and anti-diagonals).

The Backtracking Approach

The most common way to solve this is through **backtracking**, which builds the solution row by row.

1. **Start** at the first row.
2. **Try placing** a queen in each column of the current row.
3. **Check for safety:** Before placing, ensure the position isn't attacked by previously placed queens.
4. **Recurse:** If a safe spot is found, move to the next row and repeat.
5. **Backtrack:** If no safe spot exists in the current row, return to the previous row, move that queen to the next available column, and try again.

Time Complexity: $O(N!)$

Space Complexity: $O(N)$

1. **Divide:** Break the original problem into a set of smaller subproblems that are similar to the original.
2. **Conquer:** Solve the subproblems recursively. If they are small enough (base case), solve them directly.
3. **Combine:** Merge the solutions of the subproblems into the final solution for the original problem.

- **Divide:** The array is split into two halves.
- **Conquer:** Each half is sorted recursively.
- **Combine:** The two sorted halves are merged back together to form the final sorted array.

- **Binary Search:** Reduces the search space by half in each step.
- **Quick Sort:** Uses a pivot element to partition the array into smaller subarrays.
- **Merge Sort:** Recursively divides the list into halves until each contains one element.



- **Optimal Substructure:** The optimal solution to the overall problem can be constructed from the optimal solutions of its subproblems.
- **Overlapping Subproblems:** The problem can be broken down into subproblems that are reused multiple times during the calculation.

2. Conceptual Workflow

The DP process generally follows these steps:

1. **Divide:** Identify the subproblems.
2. **State Definition:** Define a "state" that captures the essential information needed to solve a subproblem.
3. **Recurrence Relation:** Formulate a mathematical relationship between the states.
4. **Compute & Store:** Solve subproblems and store their results in a data structure (array or table) to avoid redundant work.

3. Two Approaches to DP

Feature	Top-Down (Memoization)	Bottom-Up (Tabulation)
Method	Recursion + Caching	Iteration + Table filling
Logic	Solves subproblems as needed (on-demand)	Solves all possible small subproblems first
Efficiency	Can have recursion overhead (stack space)	More memory-efficient; avoids recursion overhead

The **Knapsack Problem** is a classic optimization challenge where you must fill a "knapsack" with items of specific weights and values to achieve the maximum possible profit without exceeding the weight capacity¹.

There are two primary ways to solve this depending on whether items can be broken into pieces: **Fractional (Greedy)** and **0/1 (Dynamic Programming)**.

1. Fractional Knapsack (The "Factorial" or Ratio Method)

In the **Fractional Knapsack** version, you are allowed to take fractions of an item rather than the whole thing². This is solved using a **Greedy Strategy**³.

- **Logic:** Calculate the **Profit-to-Weight ratio** (P/W) for every item⁴.
- **Step 1:** Sort all items in descending order based on their P/W ratio⁵.
- **Step 2:** Take as much of the item with the highest ratio as possible.
- **Step 3:** If the knapsack isn't full, move to the next item. If the full item doesn't fit, take the remaining capacity as a fraction of that item⁶.
- **Efficiency:** This method is highly efficient, usually $O(n \log n)$ due to sorting.

2. 0/1 Knapsack Problem

In the **0/1 Knapsack** version, you cannot break items. You either take the item (1) or leave it (0)⁷. This cannot be solved by the greedy ratio method because taking the "best" ratio might leave an awkward amount of empty space.

- **Approach:** It is solved using **Dynamic Programming**⁸.

- **Formula:** For each item i and capacity w , we decide:
 - **Exclude the item:** Value remains the same as the previous item's value at that capacity.
 - **Include the item:** Current item's value + the value of the remaining capacity from previous items.
 - **Decision:** $\text{Max}(\text{Exclude}, \text{Include})$.

The **Traveling Salesman Problem (TSP)** is a classic optimization problem in the field of Design and Analysis of Algorithms (DAA). It falls under the category of **NP-Hard** problems, meaning there is no known polynomial-time solution to solve it perfectly for all cases.

Given a set of n cities and the distance (or cost) between every pair of cities, the goal is to find the **shortest possible route** that:

In graph theory, this is defined as finding the **Minimum Weight Hamiltonian Cycle** in a complete weighted graph¹¹¹.

TSP is represented using a **Cost Matrix** or an **Adjacency Matrix**, where the element $C[i][j]$ represents the cost of traveling from city i to city j .

3. Solving Techniques in DAA

This method generates all possible permutations of cities and calculates the travel cost for each.

B. Dynamic Programming (Held-Karp Algorithm)

- **Formula:** $g(i, S) = \min \{ C_{ij} + g(j, S - \{j\}) \}$ where S is the set of unvisited cities.
- **Time Complexity:** $O(n^2 2^n)$.
- **Advantage:** Much faster than $O(n!)$ but still exponential.

C. Branch and Bound

This is a state-space search method. It uses a "lower bound" to prune (cut off) branches of the tree that cannot possibly yield a better result than the current best solution.

- **Lower Bound Calculation:** Often uses the reduced cost matrix method.
- **Efficiency:** Usually faster than DP in practice, though the worst-case remains high.

D. Greedy Approach (Nearest Neighbor)

This is a "heuristic" approach. At each city, the salesman simply moves to the nearest unvisited city.

- **Pros:** Very fast ($O(n^2)$).
- **Cons:** Does not guarantee the optimal (shortest) path; it can be very "short-sighted."

4. Example Walkthrough

Imagine 4 cities: A, B, C, D.

1. **Paths:** A-B-C-D-A, A-C-B-D-A, etc.
2. **Calculation:** If A-B(10), B-C(15), C-D(20), D-A(30), the total cost is 75.
3. **Optimization:** The algorithm compares this 75 against all other possible cycles to find the minimum.

Both **Merge Sort** and **Quick Sort** are popular sorting algorithms in DAA that utilize the **Divide and Conquer** strategy¹¹. While they share the same fundamental philosophy, their approaches to partitioning and merging differ significantly.

1. Merge Sort

Merge Sort is a stable, comparison-based algorithm that focuses on the "Combine" step². It consistently offers a time complexity of $O(n \log n)$.

- **Logic:** It repeatedly divides the array into two equal halves until each subarray contains a single element⁴.
- **Merge Step:** It then merges these subarrays in a sorted manner using an auxiliary array to store the result⁵.
- **Stability:** It is a stable sort, meaning it preserves the relative order of duplicate elements⁶.
- **Time Complexity:** $\theta(n \log n)$ for Best, Average, and Worst cases⁷.
- **Space Complexity:** $O(n)$ due to the extra space needed for the temporary merging array⁸.

2. Quick Sort

Quick Sort is an efficient, in-place sorting algorithm that focuses on the "Divide" step by using a **Pivot** element⁹.

- **Logic:** It selects a 'pivot' element and partitions the array such that elements smaller than the pivot go to the left, and larger elements go to the right¹⁰.
- **Pivot Selection:** Common strategies include picking the first, last, or a random element as the pivot¹¹.

- **In-place:** Unlike Merge Sort, it sorts the elements within the original array, making it more space-efficient¹².
- **Time Complexity:** * **Best/Average Case:** $O(n \log n)$
 - **Worst Case:** $O(n^2)$ (occurs when the pivot is always the smallest or largest element)¹⁴.
- **Space Complexity:** $O(\log n)$ for the recursive stack space¹⁵.

1. Minimum Spanning Tree (MST)

2. Prim's Algorithm

Step-by-Step Process:

Key Details:

3. Kruskal's Algorithm

Step-by-Step Process:

6. **Repeat:** Repeat steps 3-5 until there are $(V-1)$ edges in the MST¹⁹.

Key Details:

- **Complexity:** $O(E \log E)$ or $O(E \log V)$ due to sorting the edges²⁰.
- **Best For:** Sparse graphs (graphs with fewer edges)²¹.

Both the **Bellman-Ford** and **Floyd-Warshall** algorithms are used to find the shortest paths in a weighted graph, but they serve different purposes and handle different graph structures.

1. Bellman-Ford Algorithm

The Bellman-Ford algorithm is a **Single-Source Shortest Path (SSSP)** algorithm. It is more versatile than Dijkstra's because it can handle graphs with **negative edge weights**¹¹.

Step-wise Process:

- **Initialization:** Set the distance to the source vertex to 0 and all other vertices to infinity .
- **Relaxation:** Repeat the relaxation process $V - 1$ times, where V is the number of vertices³³.
- **Edge Processing:** For every edge (u, v) with weight w , if $\text{dist}(u) + w < \text{dist}(v)$, then update $\text{dist}(v) = \text{dist}(u) + w$.
- **Negative Cycle Detection:** After $V - 1$ iterations, perform one more relaxation for all edges⁵.
- **Verification:** If any distance can still be shortened during this final check, the graph contains a **negative weight cycle**, and a shortest path cannot be defined⁶.

Complexity:

- **Time Complexity:** $O(V \text{ times } E)$, where V is vertices and E is edges⁷.
- **Space Complexity:** $O(V)$ to store the distance table⁸.

2. Floyd-Warshall Algorithm

Floyd-Warshall is an **All-Pairs Shortest Path (APSP)** algorithm. It calculates the shortest distance between every possible pair of vertices in a graph using **Dynamic Programming**⁹.

Step-wise Process:

- **Matrix Setup:** Create a V times V matrix D where $D[i][j]$ is the weight of the edge from i to j .
- **Diagonal & Infinity:** Set $D[i][i] = 0$ and $D[i][j] = \text{infinity}$, if no direct edge exists between i and j
- **Iterative Triple Loop:** Use a nested loop to consider each vertex k as an intermediate point between vertices i and j
- **Comparison:** For every pair (i, j) , check if the path through k is shorter than the current direct path: $D[i][j] = \min(D[i][j], D[i][k] + D[k][j])$.
- **Final Result:** After repeating this for all vertices k (from 1 to V), the matrix contains the shortest distances between all pairs¹⁴.

Complexity:

- **Time Complexity:** $O(V^3)$, because of the three nested loops¹⁵.

- **Space Complexity:** $O(V^2)$ to maintain the distance matrix¹⁶.

Heap Sort is a comparison-based sorting algorithm that uses a **Binary Heap** data structure to organize elements¹. It is essentially an improved version of selection sort that uses a heap to find the maximum or minimum element efficiently².

1. **Build Heap:** Transform the unsorted array into a **Max-Heap** (where the parent is always greater than its children)³.
2. **Extract & Swap:** Swap the root (largest element) with the last element of the heap⁴.
3. **Heapify:** Reduce the heap size by one and "heapify" the new root to restore the Max-Heap property⁵.
4. **Repeat:** Continue until all elements are sorted⁶.

Complexity Analysis

1. P (Polynomial Time)

- **Definition:** A problem is in **P** if there exists an algorithm that can solve it in $O(n^k)$ time, where n is the input size and k is a constant.
- **Significance:** These are considered "tractable" or easy problems.
- **Examples:** Sorting (Merge Sort), searching (Binary Search), and shortest path (Dijkstra's algorithm).

2. NP (Nondeterministic Polynomial Time)

- **Definition:** A problem is in **NP** if a "certificate" (a candidate solution) can be checked in $O(n^k)$ time.
- **Key Concept:** Solving the problem might take a very long time (like exponential time), but checking a given answer is fast.
- **Example:** The **Sudoku** puzzle. Solving a blank grid is hard, but checking if a filled-out grid follows all the rules is very fast.

3. NP-Hard

- **Definition:** A problem X is **NP-Hard** if every problem in **NP** can be reduced to it in polynomial time.

- **Key Concept:** These problems do not necessarily have to be in **NP** themselves (they might not even be "verifiable" quickly).
 - **Significance:** If you find a polynomial-time solution for *any* NP-Hard problem, you have effectively solved all problems in NP in polynomial time.
 - **Example:** The **Traveling Salesman Problem (TSP)** is a classic NP-Hard problem.
-

4. NP-Complete

A problem is **NP-Complete** if it is both in **NP** and is **NP-Hard**. These are the most difficult problems in the NP class.

- **Definition:** It must be verifiable in polynomial time (NP) and any NP problem must be reducible to it (NP-Hard).
 - **Examples:** The **N-Queens Problem** and the **0/1 Knapsack Problem** (decision version)³.
-

Summary Comparison

Class	Solvable in Polynomial Time?	Verifiable in Polynomial Time?
P	Yes	Yes
NP	Maybe (Unknown)	Yes
NP-Hard	No (Likely)	Not necessarily
NP-Complete	No (Likely)	Yes